

# Technical Documentation: Cooking Recipe Assistant

## 1. Project Overview

The Cooking Recipe Assistant is a **Retrieval-Augmented Generation (RAG)** application designed to provide users with accurate cooking recipes. It combines a local vector database of recipes with a Large Language Model (LLM) to ensure that answers are grounded in real data while maintaining the conversational flexibility of an AI chef.

---

## 2. Technology Stack

### Frontend

- **HTML5 & CSS3:** Structure and styling with a modern "Chef" aesthetic.
- **JavaScript (Vanilla):** UI logic and API communication.
- **Server-Sent Events (SSE):** Real-time response streaming for a "typing" effect.

### Backend

- **Python:** Core programming language.
- **Flask:** Web framework for API endpoints ( `/api/chat` ).
- **Flask-CORS:** Handles cross-origin requests.
- **Pandas:** Data cleaning and CSV manipulation.

### AI & Vector Database

- **ChromaDB:** Open-source vector database for storing and searching recipe embeddings.
  - **Ollama:** Local runner for the LLM and Embedding models.
  - **LLM (Qwen 2.5: 1.5b):** The generative model used for creating responses.
  - **Embedding Model (nomic-embed-text):** Converts text into mathematical vectors for semantic search.
- 

## 3. Theoretical Concepts

### Retrieval-Augmented Generation (RAG)

RAG improves LLM accuracy by providing it with specific, external data. Instead of guessing, the AI "looks up" the information first.

1. **Retrieve:** Find relevant data in the database.
2. **Augment:** Add that data to the user's prompt.

3. **Generate:** Create an answer based on the combined information.

## Vector Embeddings & Semantic Search

Standard search looks for exact words. **Semantic Search** looks for *meaning*. For example, searching for "spicy pasta" might find "Arrabbiata" even if the word "spicy" isn't in the title, because their "vectors" are mathematically close.

## Basic Concepts and Definitions

- **Python:** The backend language for this project. Python scripts are executed by the Python interpreter, and third-party libraries are installed through `requirements.txt`.
- **Flask:** A lightweight web framework that turns Python functions into HTTP endpoints. It serves static files and handles the `/api/chat` request.
- **HTML:** The structure of the frontend page. It defines elements like the chat window, input box, and buttons.
- **CSS:** The style sheet that makes the app look visually consistent and readable.
- **JavaScript:** The browser-side code that sends chat queries, receives streamed responses, and updates the UI.
- **.env :** A configuration file used to store secrets and settings like `OLLAMA_BASE_URL` , `LLM_MODEL` , `EMBEDDING_MODEL` , and `TOP_K_RESULTS` .
- **Embeddings:** Numeric vectors created from text. Similar text has similar embeddings.
- **Vectors:** Lists of numbers that represent text meaning in a mathematical space.
- **ChromaDB:** The database that stores these vectors and associated recipe metadata, allowing fast similarity search.
- **Server-Sent Events (SSE):** A streaming mechanism where the server pushes data to the browser in real time. In this app, SSE is used to display the AI response token by token.
- **Prompt:** The text instructions given to the LLM. There are system prompts, user prompts, and context prompts. Together, they guide the model's behavior and output format.
- **RAG:** Combines retrieval from a database with generation from an LLM. This keeps answers grounded in actual recipe data and improves overall relevance.

---

## 4. Overall Project Flow

To explain the project simply, follow this **5-Step Flow**:

1. **User Input:** The user asks a question (e.g., "How do I make Pav Bhaji?") on the web interface.
2. **Vector Retrieval:** The backend converts that question into a vector and searches **ChromaDB** for the top 3 most similar recipes.
3. **Prompt Construction:**
  - If recipes are found, the system wraps them into a **RAG Prompt**.
  - If no match is found, it uses a **General Prompt** fallback.

4. **AI Generation:** The processed prompt is sent to **Ollama (Qwen 2.5)**. The AI reads the provided recipes and writes a response in a specific Markdown format.
  5. **Streaming Delivery:** The response is sent back to the browser bit-by-bit using **SSE**, so the user sees the answer appearing instantly.
- 

## 5. Prompt Frameworks

The project uses a structured prompt engineering approach to control the AI's behavior:

- **System Persona:** Defines the AI as an "Expert Chef Bot," ensuring it stays on topic and maintains a helpful tone.
  - **Few-Shot Formatting:** The AI is strictly instructed to follow a specific `RECIPE_OUTPUT_FORMAT` (Title, Description, Ingredients, Instructions, Chef's Tips).
  - **Conditional Logic:** The `RAG_QUERY_PROMPT` gives the AI three options:
    1. Use the database recipe exactly.
    2. Adapt the database recipe to the user's request.
    3. Fall back to internal knowledge if the database results are irrelevant.
  - **Source Attribution:** The prompt forces the AI to declare its source ( `Recipe Database` vs. `Chef's Internal Knowledge` ).
- 

## 6. Data Pipeline

1. **Data Prep ( `_01_data_prep.py` ):** Cleans the raw CSV and extracts the first 1000 rows.
  2. **Vector DB Setup ( `_02_vector_db.py` ):** Generates embeddings and populates ChromaDB.
  3. **RAG Logic ( `_03_rag.py` ):** Coordinates the search and prompt augmentation.
- 

## 7. Project Limitations

- **Dataset Size:** Only the first 1000 recipes are indexed for performance.
  - **Hardware:** Requires local CPU/GPU power to run Ollama efficiently.
  - **Hallucination:** The AI may occasionally mislabel a recipe's source if it's very confident in its internal knowledge.
  - **Static Index:** The database must be re-indexed manually if new recipes are added to the CSV.
- 

## 8. Detailed Technical Implementation

### Data Preparation and Processing

The project begins with data preparation in `_01_data_prep.py` :

- Loads the first 1000 rows from the raw `recipes.csv` file (the full dataset is 2GB, so only a subset is used for demo purposes).
- Selects only relevant columns: `title` , `ingredients` , and `directions` (renamed to `instructions` in metadata).
- Drops any rows with missing values to ensure data quality.
- Saves the cleaned data to `dataset/_02_cleaned/recipes_cleaned.csv` .

## Vector Database Setup

In `_02_vector_db.py` , the cleaned data is processed into a vector database:

- Uses **ChromaDB** as the persistent vector store.
- Employs **Ollama** with the `nomic-embed-text` model to generate embeddings.
- For each recipe, creates a document by concatenating the `title` and `ingredients` (e.g., "Pav Bhaji onions, potatoes, tomatoes...").
- Stores metadata including `title` , `ingredients` , and `instructions` for each recipe.
- Adds recipes in batches of 100 to optimize performance.
- Each recipe gets a unique ID in the format `recipe_{index}` .

## Embeddings and Vectors

- **Embeddings** are dense vector representations of text that capture semantic meaning. They convert human-readable text into numerical arrays (vectors) that machines can process.
- **Vectors** are fixed-length arrays of floating-point numbers (e.g., 768 dimensions for `nomic-embed-text` ).
- The embedding model ( `nomic-embed-text` ) transforms recipe text into vectors, allowing semantic similarity searches rather than exact keyword matches.
- During search, the user's query is also embedded into a vector, and cosine similarity is used to find the closest recipe vectors in the database.

## Retrieval-Augmented Generation (RAG) Process

The RAG logic in `_03_rag.py` handles the core retrieval and augmentation:

- **Top-K Results:** Retrieves the top 3 most similar recipes ( `TOP_K_RESULTS = 3` ) based on vector similarity.
- **Distance Threshold:** Filters results with a similarity score below 1.0 (higher scores indicate less similarity; values  $> 1.0$  are discarded as irrelevant).
- If relevant recipes are found, formats them into a structured context string with title, ingredients, and instructions.
- Ingredients and instructions are parsed and formatted as bullet points or numbered lists for readability.

## Data Flow into the LLM

1. **User Query Reception:** The query is received via the `/api/chat` endpoint in `app.py`.
2. **Query Cleaning:** Removes conversational fluff (e.g., "please", "hey") using `clean_query()` to improve search relevance.
3. **Embedding and Search:** The cleaned query is embedded and used to query ChromaDB for the top 3 similar recipes.
4. **Prompt Construction:**
  - If context is found (at least one recipe within the distance threshold), uses `RAG_QUERY_PROMPT` with the retrieved recipes inserted as `{context}`.
  - If no relevant context, falls back to `GENERAL_QUERY_PROMPT`.
5. **LLM Interaction:** Sends the constructed prompt to Ollama's Qwen 2.5 model along with the system prompt.
  - System prompt defines the AI as an "Expert Chef Bot" and enforces output format.
  - The LLM generates a response in the specified Markdown format.
6. **Streaming Response:** The response is streamed back to the frontend using Server-Sent Events (SSE) for real-time display.

## Key Configuration Parameters

- **Top-K Results:** 3 (configurable via `TOP_K_RESULTS` environment variable).
- **Distance Threshold:** 1.0 (fixed; higher values allow more inclusive results).
- **Embedding Model:** `nomic-embed-text` (via Ollama).
- **LLM Model:** `qwen2.5:1.5b` (via Ollama).
- **Batch Size for DB Population:** 100 recipes per batch.

## Model Selection and Why These Models

- **nomic-embed-text** is chosen for embeddings because it is optimized for semantic similarity and is available locally through Ollama. It produces compact vectors that capture the meaning of recipe titles and ingredients well, which makes the ChromaDB search more reliable than plain keyword matching.
- **qwen2.5:1.5b** is used as the LLM because it balances response quality, speed, and local resource usage. It is smaller than very large models but still capable of generating coherent recipe instructions and following structured prompts. This makes it a practical choice for local deployment without requiring massive hardware.
- **Why not use larger models?** Larger models can be more accurate but require much more RAM and slower response times on local machines. This project prioritizes a responsive local experience, so a mid-sized model is preferred.
- **Why not use only internal LLM knowledge?** The RAG design uses the vector database to ground answers in actual recipes. This helps avoid hallucinations and improves relevance compared to asking the LLM to generate recipes from scratch.

## Additional Technical Details

- **Collection Management:** ChromaDB collection is initialized once at app startup and reused for all queries.
  - **Error Handling:** Checks for Ollama connectivity and model availability before proceeding.
  - **Metadata Storage:** Full recipe details are stored in metadata to avoid re-embedding during retrieval.
  - **Prompt Engineering:** Uses structured prompts to ensure consistent output format and source attribution.
  - **Performance Considerations:** Limited to 1000 recipes to keep embedding and search times reasonable on local hardware.
- 

## 9. Function Explanations

This section provides detailed explanations of all key functions in the codebase, organized by file.

### Data Preparation ( `backend/src/_01_data_prep.py` )

- `clean_data(df)` :
  - **Purpose:** Cleans and filters the raw recipe dataset for use in the vector database.
  - **Parameters:** `df` (pandas DataFrame) - The loaded CSV data.
  - **Process:** Selects only `title`, `ingredients`, and `directions` columns; drops rows with missing values; saves cleaned data to `dataset/_02_cleaned/recipes_cleaned.csv`.
  - **Return:** None (saves to file).

### Vector Database Setup ( `backend/src/_02_vector_db.py` )

- `create_and_populate_db(df)` :
  - **Purpose:** Initializes ChromaDB collection and populates it with embedded recipe vectors.
  - **Parameters:** `df` (pandas DataFrame) - The cleaned recipe data.
  - **Process:** Creates or wipes the "recipes" collection; generates embeddings for each recipe (title + ingredients); stores documents, metadata (title, ingredients, instructions), and IDs; adds in batches of 100 for efficiency.
  - **Return:** ChromaDB collection object.

### RAG Logic ( `backend/src/_03_rag.py` )

- `format_list_string(s)` :
  - **Purpose:** Parses and formats string representations of lists (e.g., ingredients or instructions) into readable bullet points or numbered steps.
  - **Parameters:** `s` (str) - The raw string from the dataset.
  - **Process:** Attempts JSON parsing; if it's a list, joins items; otherwise, splits by sentences or periods; formats as bullets or numbered lists.

- **Return:** Formatted string (e.g., "- Item 1\n- Item 2").
- **search\_recipes(collection, query: str, n\_results: int = TOP\_K\_RESULTS) :**
  - **Purpose:** Searches the vector database for recipes similar to the user's query.
  - **Parameters:** `collection` (ChromaDB collection) - The recipe database; `query` (str) - The cleaned user query; `n_results` (int) - Number of results to retrieve (default: 3).
  - **Process:** Queries ChromaDB with embedded query; filters results by distance threshold (1.0); formats relevant recipes into context string with title, ingredients, and instructions.
  - **Return:** Tuple of (context string or None, minimum distance score).

## Utilities ( backend/utils/functions.py )

- **get\_ollama\_embedding\_function() :**
  - **Purpose:** Creates and validates the Ollama embedding function for ChromaDB.
  - **Parameters:** None.
  - **Process:** Loads Ollama URL and model from environment; checks if model is available on Ollama; returns ChromaDB OllamaEmbeddingFunction.
  - **Return:** Embedding function object.
  - **Raises:** RuntimeError if Ollama is unreachable or model not found.
- **get\_recipe\_collection() :**
  - **Purpose:** Retrieves or creates the ChromaDB collection for recipes.
  - **Parameters:** None.
  - **Process:** Initializes ChromaDB client; gets or creates "recipes" collection with embedding function; prints collection count.
  - **Return:** ChromaDB collection object.
- **clean\_query(query: str) -> str :**
  - **Purpose:** Preprocesses user queries for better vector search by removing conversational fluff.
  - **Parameters:** `query` (str) - The raw user input.
  - **Process:** Strips whitespace; removes common words like "please", "hey", "bot" that don't add semantic value.
  - **Return:** Cleaned query string.

## Main Application ( backend/app.py )

- **index() :**
  - **Purpose:** Serves the main HTML page for the frontend.
  - **Parameters:** None (Flask route).
  - **Process:** Returns `index.html` from the frontend directory.
  - **Return:** Flask response with HTML file.



- **`static_files(filename)`** :
    - **Purpose:** Serves static files (CSS, JS) for the frontend.
    - **Parameters:** `filename` (str) - The requested file path.
    - **Process:** Returns the file from the frontend directory.
    - **Return:** Flask response with the file.
  - **`chat()`** :
    - **Purpose:** Handles the chat API endpoint, processing user queries and streaming LLM responses.
    - **Parameters:** None (Flask route, gets data from request).
    - **Process:** Validates input; cleans query; searches for recipes; constructs appropriate prompt; streams response from Ollama via SSE.
    - **Return:** Flask Response object with SSE stream.
    - **Inner Function:** `generate()` - Yields SSE-formatted chunks from Ollama's streaming response.
- 

## 10. Setup and Installation

### Prerequisites

- **Python 3.8+:** Required for running the backend.
- **Ollama:** Local AI model runner. Download from [ollama.ai](https://ollama.ai) and install.
- **Git:** For cloning the repository (if applicable).

### Installation Steps

#### 1. Clone or Download the Project:

- Place the project folder in your desired location (e.g.,  
`C:\Users\tiyaj\OneDrive\Desktop\cooking_recipe_assistant`).

#### 2. Install Python Dependencies:

- Navigate to the project root.
- Run: `pip install -r requirements.txt`
- Dependencies include: `pandas` (data processing), `chromadb` (vector database), `ollama` (AI client), `python-dotenv` (environment variables), `flask` (web framework), `flask-cors` (CORS handling).

#### 3. Install Ollama Models:

- Start Ollama: `ollama serve`
- Pull required models:
  - `ollama pull nomic-embed-text` (for embeddings)



- `ollama pull qwen2.5:1.5b` (for LLM; note: `.env.example` shows `llama3`, but code uses `qwen2.5:1.5b`)

#### 4. Configure Environment:

- Copy `.env.example` to `.env`.
- Update variables as needed (see Configuration section below).

#### 5. Prepare Data:

- Ensure `dataset/_01_raw/recipes.csv` exists (original dataset with columns: title, ingredients, directions).
- Run data preparation: `python backend/src/_01_data_prep.py`
- Run vector DB setup: `python backend/src/_02_vector_db.py`

#### 6. Run the Application:

- Start the backend: `python backend/app.py`
- Open `frontend/index.html` in a browser or serve it via a local server.

## Running the Full Pipeline

- Execute scripts in order: `_01_data_prep.py` → `_02_vector_db.py` → `app.py`
  - The app runs on `http://localhost:5000` by default.
- 

## 11. Configuration

The project uses environment variables stored in `.env` for configuration:

- **OLLAMA\_BASE\_URL**: URL for Ollama server (default: `http://localhost:11434`).
- **LLM\_MODEL**: Model for generating responses (e.g., `qwen2.5:1.5b`).
- **EMBEDDING\_MODEL**: Model for creating embeddings (e.g., `nomic-embed-text`).
- **TOP\_K\_RESULTS**: Number of top similar recipes to retrieve (default: 3).

Modify `.env` to change models or parameters. Restart the app after changes.

---

## 12. Dataset Details

- **Source**: The dataset is a subset of a larger recipe database (full size: ~2GB).
- **Structure**:
  - **Columns**: `title` (str), `ingredients` (str, often JSON-like list), `directions` (str, step-by-step instructions).
  - **Sample Row**: Title: "Pav Bhaji", Ingredients: "['onions', 'potatoes', 'tomatoes']", Directions: "1. Boil potatoes... 2. Mash and mix..."

- **Processing:** Only first 1000 rows are used; cleaned to remove nulls; ingredients/directions parsed for formatting.
  - **Storage:** Cleaned data in `dataset/_02_cleaned/recipes_cleaned.csv` ; embedded in `chroma_db/` directory.
- 

## 13. API Endpoints

The backend exposes the following endpoints via Flask:

- **GET /** : Serves the main HTML page ( `frontend/index.html` ).
  - **GET /<path:filename>** : Serves static files (CSS, JS) from `frontend/` .
  - **POST /api/chat** : Main chat endpoint.
    - **Request Body:** `{"query": "user question"}`
    - **Response:** Server-Sent Events stream with LLM response.
    - **Error Handling:** Returns `{"error": "message"}` for invalid queries.
- 

## 14. Troubleshooting

### Common Issues

- **Ollama Not Running:** Error: "Could not reach Ollama". Solution: Run `ollama serve` and ensure models are pulled.
- **Model Not Found:** Error: "Embedding model not found". Solution: `ollama pull <model_name>` .
- **No Recipes Found:** Check if data prep and vector DB scripts were run; verify `chroma_db/` exists.
- **CORS Errors:** Ensure Flask-CORS is installed; browser may block local requests.
- **High Memory Usage:** Limit dataset size or use smaller models.
- **Slow Responses:** Reduce `TOP_K_RESULTS` or use GPU acceleration in Ollama.

### Debugging Tips

- Check console logs in `app.py` for debug prints (e.g., context found/not found).
  - Verify `.env` variables are loaded correctly.
  - Test Ollama directly: `ollama run qwen2.5:1.5b` with a prompt.
- 

## 15. Performance and Optimization

- **Embedding Time:** ~10-30 seconds for 1000 recipes (depends on hardware).
- **Search Latency:** <1 second per query on local hardware.
- **Memory Usage:** ChromaDB stores vectors in SQLite (~50-100MB for 1000 recipes).

- **Optimizations:**
    - Batch processing (100 recipes at a time) during DB population.
    - Distance threshold (1.0) filters irrelevant results.
    - Streaming responses reduce perceived latency.
  - **Bottlenecks:** Ollama model size; increase RAM/GPU for larger datasets.
- 

## 16. Future Enhancements

- **Scalability:** Support full 2GB dataset; implement pagination for results.
  - **UI Improvements:** Add recipe images, favoriting, or advanced search filters.
  - **Model Upgrades:** Experiment with larger LLMs or fine-tuned models for recipes.
  - **Security:** Add input sanitization; rate limiting for API.
  - **Deployment:** Containerize with Docker; deploy to cloud with persistent storage.
  - **Testing:** Add unit tests for functions; integration tests for RAG pipeline.
  - **Multilingual Support:** Handle non-English recipes and queries.
- 

## 17. Frontend Details

The frontend is a single-page application built with vanilla JavaScript, HTML, and CSS, providing a real-time chat interface for interacting with the AI assistant.

### HTML Structure ( `frontend/index.html` )

- **Layout:** Uses a sidebar for navigation (currently just "Chat") and a main area for the chat interface.
- **Chat Container:** Displays messages in bubbles with avatars (AI for bot, user for human).
- **Input Area:** Text input field with a send button (SVG icon).
- **Metadata:** Includes meta tags for SEO, links to Google Fonts (Inter), and the CSS file.
- **Initial Message:** Pre-loads a welcome message from the AI bot.

### JavaScript Logic ( `frontend/script.js` )

- **State Management:** Tracks `isStreaming` to prevent multiple simultaneous requests.
- **`sendMessage()` Function:**
  - Retrieves user input, validates it, and appends a user message bubble.
  - Shows a typing indicator while processing.
  - Sends a POST request to `/api/chat` with JSON payload `{"query": "user input"}`.
  - Handles the Server-Sent Events (SSE) stream: Parses incoming tokens, accumulates text, and renders Markdown using `marked.js`.
  - Updates the bot message bubble in real-time as tokens arrive.

- Handles errors (e.g., server down) by displaying error messages.
- **Message Handling:** `appendMessage()` creates and styles message divs with fade-in animation.
- **UI Helpers:** `showTypingIndicator()`, `toggleInput()` (disables input during streaming), `scrollToBottom()` for auto-scrolling.
- **Event Listeners:** Enter key press triggers send; input focus management.

## CSS Styling ( `frontend/style.css` )

- **Theme:** Dark mode with black backgrounds, white text, and subtle borders.
- **Variables:** CSS custom properties for colors, fonts, and dimensions (e.g., `--bg-primary`, `--sidebar-width`).
- **Layout:** Flexbox for sidebar and main content; responsive design.
- **Animations:** Fade-in for messages; hover effects on buttons.
- **Typography:** Uses Inter font; consistent spacing and sizing.
- **Accessibility:** High contrast for readability; focus states for inputs.

## Key Features

- **Real-Time Streaming:** Messages appear word-by-word as the LLM generates them.
  - **Markdown Rendering:** Converts LLM output (e.g., recipe formats) to HTML for proper display.
  - **Error Handling:** Graceful fallbacks for network issues or invalid responses.
  - **Responsive:** Adapts to different screen sizes (though optimized for desktop).
- 

# 18. Flask Integration and Connections

Flask serves as the backend server, handling HTTP requests, serving static files, and managing the API. It connects the frontend, vector database, and LLM seamlessly.

## Server Setup ( `backend/app.py` )

- **Initialization:** Loads environment variables; initializes ChromaDB collection on startup.
- **CORS Handling:** `Flask-CORS` allows cross-origin requests from the frontend.
- **Routes:**
  - `GET /`: Serves `frontend/index.html`.
  - `GET /<path:filename>`: Serves static files (CSS, JS) from `frontend/`.
  - `POST /api/chat`: Main API endpoint for chat queries.

## API Endpoint Flow ( `/api/chat` )

1. **Request Handling:** Receives JSON payload with `query`; validates input.
2. **Query Processing:** Cleans the query using `clean_query()`.
3. **Retrieval:** Calls `search_recipes()` to get relevant context from ChromaDB.

4. **Prompt Construction:** Builds the appropriate prompt (RAG or general) with context.
5. **LLM Interaction:** Sends prompt to Ollama via chat API; streams tokens.
6. **Response Streaming:** Yields SSE events ( `data: {"token": "..."}`  ) to the frontend.
7. **Error Responses:** Returns JSON errors for invalid queries.

## Connections Overview

- **Frontend ↔ Flask:** HTTP requests (GET for files, POST for chat); SSE for streaming responses. Runs on `http://localhost:5000`.
- **Flask ↔ ChromaDB:** Python client connection; queries collection for embeddings and metadata.
- **Flask ↔ Ollama:** HTTP client to `OLLAMA_BASE_URL` (default `localhost:11434`); sends prompts and receives streamed tokens.
- **Data Flow:** User input → Frontend → Flask → ChromaDB/Ollama → Flask → SSE → Frontend → UI update.
- **Dependencies:** `Flask` for web server; `requests` implicit via Ollama client; `chromadb` for vector ops.
- **Concurrency:** Single-threaded by default; handles one request at a time (streaming allows perceived parallelism).
- **Security:** Local-only; no authentication; CORS enabled for browser access.

## Key Integration Points

- **Static Serving:** Flask serves the entire frontend, making it a full-stack app in one process.
- **Streaming:** Uses `Response` with generator to yield SSE data, enabling real-time UI updates.
- **Environment Config:** All connections (Ollama, DB paths) configured via `.env` for flexibility.